Independent University, Bangladesh

CSE 313 (Compiler Construction) Midterm Summer 2023 · Sec: 1 · Total: 20 · Time: 1h20m

Instructor: Bijoy Rahman Arif. Answer all 4. Marks are after each question.

Question paper

1. [CO1] Consider the following grammar. Is the grammar Ambiguous? If so, find a string for which it is Ambiguous and finally, convert to Unambiguous grammar without changing the original Language. Find the Operator Precedence and Associativity of Unambiguous grammar. If the Unambiguous grammar is Left Recursive and/ or Non-deterministic, convert the grammar to Right Recursive and Deterministic as well. **[5]**

```
S → A @ S | A
A → A & B | B
B → B # B | B ~ C | C
C → i
```

2. [CO3] Consider the following grammar, and try to parse the string `( ( ( (` using all the conversion and steps in LL(1) parsing. Where variable = { S } and terminal = { (, ϵ }. If parsing is not possible, point out why it is not possible. **[5]**

```
S → ( ( b
S → ε
```

3. [CO3] An ambiguous grammar is Non-deterministic Grammar (NDG). After converting it to Deterministic Grammar it is still ambiguous. What is the reason behind this? Is it possible to prove: "Determinism does not change ambiguity of a grammar." Justify with reasonings. **[5]**

4. [CO1] Point out TRUE/ FALSE of the following statements: **[5]**

(a) Left Recursive Grammar (LRG) can not be used in LL(1).

(b) Non-deterministic Grammar (NDG) can not be used in LL(1).

(c) Recursive Descent Parser (RDP) is as powerful as LL(1)

(d) Recursive Descent Parser (RDP) uses OS stack.

(e) Recursive Descent Parser (RDP) can use Non-deterministic Grammar (NDG).

Answers (exam layout, with comments)

Answer 1 [5]

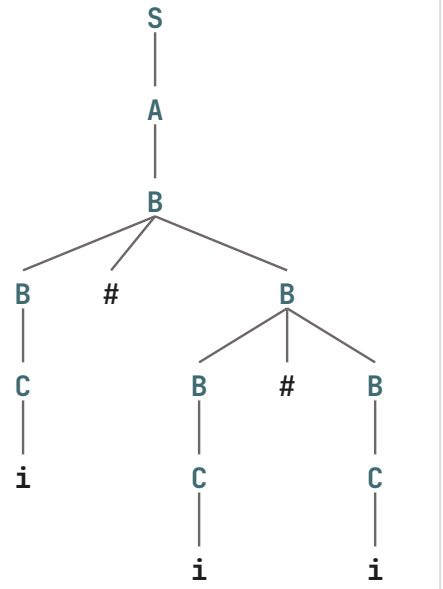
`WRITE THIS`

Yes, the grammar is ambiguous. `B → B # B` has B on both sides of #. String: `i # i # i`.

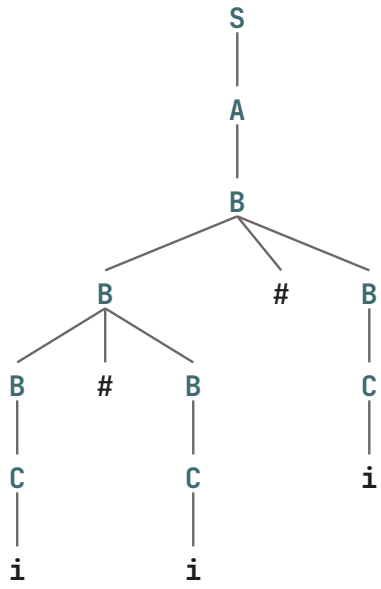
LMD 1: S ⇒ A ⇒ B ⇒ B # B ⇒ C # B ⇒ i # B ⇒ i # B # B ⇒ i # C # B ⇒ i # i # B ⇒ i # i # C ⇒ i # i # i

LMD 2: S ⇒ A ⇒ B ⇒ B # B ⇒ B # B # B ⇒ C # B # B ⇒ i # B # B ⇒ i # C # B ⇒ i # i # B ⇒ i # i # C ⇒ i # i # i

Tree 1



Tree 2



Since for the string `i # i # i` there exists more than one leftmost derivation parse tree, the grammar is ambiguous.

[Assuming ~ has higher precedence than #, and both # and ~ are left associative]

Unambiguous grammar (same language, # and ~ get their own levels):

```
S → A @ S | A
A → A & B | B
B → B # D | D
D → D ~ C | C
C → i
```

Operator	Precedence	Associativity	Why
@	1 (lowest)	right	S → A @ S
&	2	left	A → A & B
#	3	left	B → B # D
~	4 (highest)	left	D → D ~ C

It is left recursive (A, B, D) and non-deterministic (S → A @ S | A). Convert both.

LRG → RRG

A → A & B | B: α = `& B`, β = `B`.

```
A → B A'
A' → & B A' | ε
```

B → B # D | D: α = `# D`, β = `D`.

```
B → D B'
B' → # D B' | ε
```

D → D ~ C | C: α = `~ C`, β = `C`.

```
D → C D'
D' → ~ C D' | ε
```

Final RRG:

```
S → A @ S | A
A → B A'
A' → & B A' | ε
B → D B'
B' → # D B' | ε
D → C D'
D' → ~ C D' | ε
C → i
```

Rule: A → Aα | β ⇒ A → βA', A' → αA' | ε. "β goes first, α goes to the new guy, new guy ends with ε."

NDG → DG

Round 1: in S, `A @ S`, `A` all start with `A`. Take it out once:

```
S → A S'
S' → @ S | ε
```

Final DG:

```
S → A S'
S' → @ S | ε
A → B A'
A' → & B A' | ε
B → D B'
B' → # D B' | ε
D → C D'
D' → ~ C D' | ε
C → i
```

> **Lose marks by:** stopping after the unambiguous grammar (this question has four parts); "fixing" S → A @ S, which is already right recursive.

Answer 2 [5]

WRITE THIS

Conversions first. Left recursion? No, S does not start with S. Non-determinism? No, only one alternative starts with a terminal. So no conversion is needed. b is not in the given terminal set; treat it as a terminal, as printed.

Numbered rules: 1. S → ((b 2. S → ε

Step 1 · FIRST & FOLLOW

Non-terminal	FIRST	FOLLOW
S	{ (, ε }	{ $ }

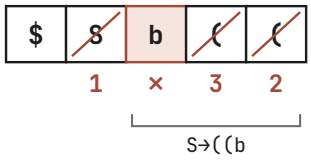
Step 2 · LL(1) parsing table

	(	b	$
S	S→((b		S→ε

No cell has more than one rule ∴ **the grammar is LL(1)**.

Step 3 · Parse `( ( ( (` (horizontal stack)

Buffer: `( ( ( ( $` LA ↑ starts at `(`



How to read it: red number under a cell = the order it was popped. Bracket = what that pop pushed (right side, reversed). →ε above a cell = it was replaced by ε (push nothing). × = where parsing stopped.

Step labels (write these under the cells on paper):

```
1. pop S (LA = (): S→((b, push b ( (      err. Top = b (terminal) but LA = (:
2. match (, pop it, move LA → (           they do not match → error
3. match (, pop it, move LA → (
```

Parsing is not possible. Reason: Top = b (terminal) but LA = (: they do not match → error. ∴ error, the string `( ( ( (` is not in the language.

▼ How the FIRST / FOLLOW were found (rough work, not needed on paper)

- `FIRST(S): S → ( ( b gives { ( }. S → ε gives ε. Total { (, ε }.`
- `FOLLOW(S): start symbol → add $. Total { $ }.`

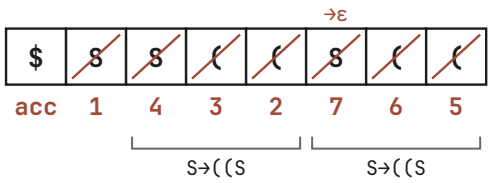
Extra reason: the grammar makes only two strings, `((b` and ε, so `((((` can never be accepted.

If the paper meant S → ((S | ε (b is a typo for S)

Write this second reading too. It costs one minute and protects your marks.

Parse `( ( ( (` with S → ((S | ε

Buffer: `( ( ( ( $` LA ↑ starts at `(`

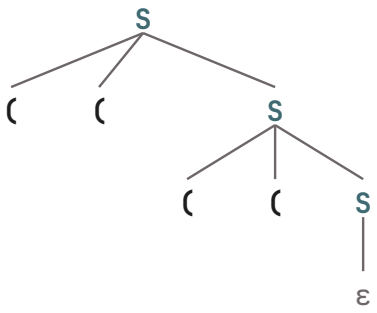


How to read it: red number under a cell = the order it was popped. Bracket = what that pop pushed (right side, reversed). →ε above a cell = it was replaced by ε (push nothing). × = where parsing stopped.

Step labels (write these under the cells on paper):

```
1. pop S (LA = (): S→((S, push S ( (      5. match (, pop it, move LA → (
2. match (, pop it, move LA → (           6. match (, pop it, move LA → $
3. match (, pop it, move LA → (           7. pop S (LA = $): S→ε, push nothing
4. pop S (LA = (): S→((S, push S ( (      acc. Top = $ and LA = $ → accept
```

Parse tree



Leaves left to right (skip ε): `( ( ( (` = the input. Stack top $ and LA $ ∴ the string is accepted, it is syntactically correct.

> **Lose marks by:** writing only "error". Write the top, the LA and why.

Answer 3 [5]

`WRITE THIS`

Take `E → E + E | E * E | id`. It is ambiguous, and it is an NDG (two alternatives start with E). Left factor it:

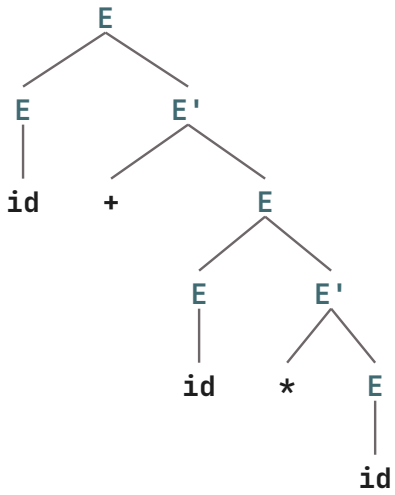
```
E → E E' | id
E' → + E | * E
```

The string `id + id * id` still has two leftmost derivations:

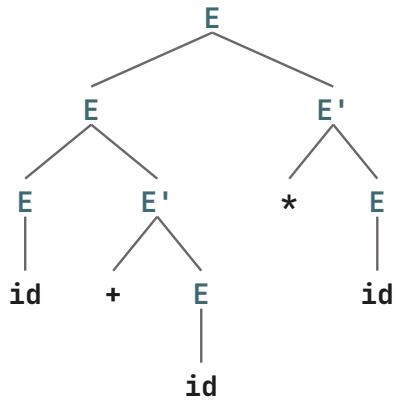
LMD 1: E ⇒ E E' ⇒ id E' ⇒ id + E ⇒ id + E E' ⇒ id + id E' ⇒ id + id * E ⇒ id + id * id

LMD 2: E ⇒ E E' ⇒ E E' E' ⇒ id E' E' ⇒ id + E E' ⇒ id + id E' ⇒ id + id * E ⇒ id + id * id

Tree 1



Tree 2



Reason: left factoring only removes common prefixes. It delays the choice between alternatives; it does not remove any alternative or any tree.

Proof (one-to-one mapping): each use of $A \to \alpha\beta_i$ in the old grammar becomes $A \to \alpha A'$ then $A' \to \beta_i$ in the new one, and back. So each parse tree maps to exactly one parse tree, and every string keeps the same number of trees.

∴ Yes, it can be proved: determinism does not change ambiguity. To remove ambiguity you must restructure the grammar with precedence levels.

> **Lose marks by:** giving only the claim with no example; no derivations for the factored grammar.

Answer 4 [5]

WRITE THIS

#	Answer	Reason (one line)
(a)	**True**	LL(1) is top-down; left recursion makes it loop and puts two rules in one cell.
(b)	**True**	Two alternatives with the same prefix fall in the same cell (FIRST/FIRST conflict).
(c)	**False**	RDP with backtracking can try every alternative, so it accepts grammars LL(1) cannot. More general, but slower.
(d)	**True**	One procedure per non-terminal, called recursively, so it uses the program (OS) call stack.
(e)	**True**	With backtracking it tries one alternative, and on failure goes back and tries the next.